

杭州电子科技大学

本科毕业设计（论文）

(2026 届)

基于大模型的中小學生智能编程学习平台设计与实现

站点名称:	编程教育平台
专 业:	计算机科学与技术
班 级:	22052323
学 号:	22010210
学生姓名:	林柏伟
指导教师:	匡振中
完成日期:	2026 年 5 月

诚信承诺

我谨在此承诺：本人所写的毕业论文《基于大模型的中小学生智能编程学习平台设计与实现》均系本人独立完成，没有抄袭行为，凡涉及其他作者的观点和材料，均作了注释，若有不实，后果由本人承担。

承诺人（签名）：_____

年 月 日

摘要

大语言模型在代码生成，与错误诊断上的可用性，已获验证。自然语言交互、个性化反馈，两端尤为突出。当前编程教学，在资源可得性、辅导时效、过程追踪，和即时反馈上，仍有短板。入门阶段，兴趣滑坡并不少见。本文据此，设计了一套面向中小学生的智能编程学习平台，在因材施教与教学效率之间，寻找折中。

架构上前后端分离。交互界面，基于 Nuxt 3 与 Vue 3 搭建，样式依托 Tailwind CSS、DaisyUI。核心业务，由 FastAPI、SQLAlchemy 承担。数据层选用 SQL Server 持久化，Redis 缓存加速，阿里云 OSS 存放教学资源。

功能覆盖用户认证、课程编排、在线编程实践、任务管理、师生互动，及教学资源管理。大模型驱动的辅学模块，承接自由问答、代码逻辑拆解、编译错误分析、问题分解引导，以及抽象建模与算法设计指导。针对中小学生认知节奏，提示词策略、输出语气，和错误反馈均做了调整，反馈逐步给出，而非一次倾倒。

全文围绕需求分析、总体设计、核心模块实现与功能验证展开。平台已为学生日常编程练习，与教师教学管理，提供较为完整的操作支撑，在代码稳定运行、错误反馈、进度跟踪与资源组织方面，已有初步实践基础。后续可在 RAG 私域知识增强、认知模板评估，及更细粒度的个性化推荐方向，继续拓展。

关键词：大语言模型；编程教育；智能辅学；在线编程；教学平台

ABSTRACT

The availability of large language models in code generation and error diagnosis has been verified. Natural language interaction and personalized feedback are particularly prominent at both ends. In the current programming teaching, there are still shortcomings in resource availability, counseling timeliness, process tracking, and instant feedback. In the introductory stage, interest landslides are not uncommon. Based on this, **this paper** designs a set of intelligent programming learning platform for primary and middle school students, and finds a compromise between teaching students in accordance with their aptitude and teaching efficiency.

The front and back ends are separated on the architecture. The interactive interface is built based on Nuxt 3 and Vue 3, and the style relies on Tailwind CSS and DaisyUI. The core business is undertaken by FastAPI and SQLAlchemy. The data layer uses SQL Server persistence, Redis cache acceleration, and Alibaba Cloud OSS to store teaching resources.

The functions cover user authentication, course arrangement, online programming practice, task management, teacher-student interaction, and teaching resource management. The large model-driven auxiliary module undertakes free question answering, code logic disassembly, compilation error analysis, problem decomposition guidance, and abstract modeling and algorithm design guidance. In view of the cognitive rhythm of primary and middle school students, the prompt word strategy, output tone, and error feedback have been adjusted, and the feedback is gradually given, rather than a dumping.

The full text focuses on demand analysis, overall design, core module implementation and functional verification. The platform has provided a relatively complete operational support for students' daily programming exercises and teachers' teaching management. It has a preliminary practical basis in terms of stable code operation, error feedback, progress tracking and resource organization. Subsequently, it can continue to expand in the RAG private domain knowledge enhancement, cognitive template evaluation, and more fine-grained personalized recommendation direction.

Key Words: large language model; programming education; intelligent tutoring; online programming; learning platform

目 录

1	引言	1
1.1	研究背景	1
1.2	研究意义	1
1.3	国内外研究现状	1
1.4	研究内容与目标	2
1.5	论文结构安排	2
2	概述	3
2.1	系统建设目标概述	3
2.2	大语言模型与智能辅学	3
2.3	前端开发技术	3
2.4	后端开发技术	4
2.5	数据存储与资源管理技术	4
2.6	在线编程与安全执行技术	4
2.7	系统需求分析	4
3	系统总体设计	9
3.1	系统设计目标	9
3.2	系统总体架构设计	9
3.3	系统功能模块设计	9
3.4	前后端结构设计	9
4	系统详细设计	12
4.1	数据库设计	12
4.2	智能辅学模块设计	14
4.3	系统部署设计	16
5	软件实现	17
5.1	用户认证与权限控制实现	17
5.2	课程与知识点管理功能实现	17
5.3	在线编程与代码运行功能实现	18
5.4	大模型辅助教学功能实现	19
5.5	师生互动与消息功能实现	21

5.6	教学资源管理功能实现	22
5.7	学习行为记录与事件追踪功能实现	22
5.8	系统界面展示	22
6	系统测试与调试	26
6.1	测试环境	26
6.2	功能测试	26
6.3	前端构建与代码质量检查	28
6.4	性能测试	29
6.5	安全测试	30
6.6	测试结果分析	31
7	结论	33
	参考文献	34
	致谢	37
	附录	38

1 引言

1.1 研究背景

编程教育逐渐进入中小学信息素养培养体系。编程学习并不只要求学生掌握语法规则，还涉及逻辑推理、抽象建模和问题求解等能力的形成。对初学者而言，语法理解、报错定位和任务分解往往是连续出现的难点；教师在有限课堂时间内，也难以对每名学生的卡点进行持续反馈^[1]。

大语言模型能读懂自然语言、解释代码并与人对话，这几个长板让它在编程学习平台里有了用武之地。LLM 代码生成、调试辅助、形成性反馈和个性化学习支持方面的研究已有相当积累^[2-3]。但当场景切换到中小学课堂，回答的准确性、适龄性和可控性就变得格外苛刻——通用聊天能力不经约束地塞进教学流程是行不通的。

基于这一问题，本文并不把研究重点放在单一大模型接口调用上，而是面向中小學生编程学习流程，设计并实现一套集课程学习、在线编程、智能辅学、教师干预和过程记录于一体的平台。平台以“教学约束、风险兜底、过程留痕”为设计前提，使智能能力嵌入具体学习环节，而不是脱离课堂情境单独运行。

1.2 研究意义

现有 LLM 辅助编程教育的研究较多聚焦于高等教育、单一任务或工具效果评估；面向中小學生，且同时覆盖课程学习、在线编程、错误诊断、教师干预和过程记录的平台化研究仍相对不足^[3-4]。本文从系统实现与教学适配两个角度展开，可为同类平台建设提供参考。

本平台还将大模型能力嵌入日常学习流程，而不是把它作为孤立问答工具使用。Tang 等研究指出，在教学设计合理的前提下，LLM 支持能够提升学生编程表现、学习兴趣与编程自我效能感^[5]。本文进一步强调教师可控、过程可追踪和输出可约束，使大模型承担课堂中的辅助支架角色，而非替代教师的答案来源。

1.3 国内外研究现状

国内外关于大模型与编程教育结合的研究，主要集中在代码生成、代码解释、错误诊断、形成性反馈和学习行为分析等方向。Wang 等梳理出 LLM 在助学、助教和自适应学习上的潜力，也挑了可靠性、偏差和伦理风险的刺^[2]。Raihan 等对计算机科学教育的综述显示，LLM 已渗入编程学习、代码理解和课程支持，但学效和教学适配怎么落地，

还需要更多实践来撑^[3]。Pitts 等盯得更紧，直言真正起效的编程教育应用，通常离不开教师参与和人在回路^[6]。

K-12 场景的研究给本文提供了更贴近的参照。Tang 等在中国初中阶段的实验中看到，LLM 介入之后，学生的编程表现、学习兴趣和自我效能感都有改善^[5]。Chen 等的 ChatScratch 把 AI 增强机制嵌入可视化编程环境，儿童自主学习和作品质量出现了积极变化^[7]。Ma 等则从调试教学切入，主张 LLM 更适合做协同型辅助，通过引导定位错误、解释修改理由来推动调试能力^[8]。

已有研究也划出了应用边界。Gonzalez-Maldonado 等的实验表明，光靠提示工程，LLM 很难稳当地完成 K-12 块编程项目的生成和自动评分^[9]。Pirzado 等则点出了代码理解、调试支持、学习依赖和反馈可信性上的风险^[10]。这些边界提醒我们：平台不能退化成外接聊天窗口，基础教学管理、教师干预、过程记录 and 智能辅学功能缺一不可。

1.4 研究内容与目标

本文以“基于大模型的中小学生智能编程学习平台设计与实现”为研究主题，主要完成以下工作。

- (1) 设计并实现面向学生与教师的编程学习平台。
- (2) 构建课程、知识点、任务、资源和行为记录等核心业务模块。
- (3) 集成大模型能力，实现代码解释、报错分析、问题分解、抽象建模和算法设计引导。
- (4) 实现在线代码运行与安全控制机制。
- (5) 对系统功能与运行效果进行测试分析。

1.5 论文结构安排

本文共分为七章。第一章为引言，说明研究背景、研究意义、国内外研究现状和论文结构。第二章为概述，介绍系统建设目标、关键技术和需求分析。第三章为总体设计，给出系统设计目标、总体架构、功能模块和前后端结构。第四章为系统详细设计，说明数据库、智能辅学模块和部署方案。第五章为软件实现，介绍主要功能模块的实现过程。第六章为系统测试与调试，对平台关键流程进行验证。第七章为结论，总结全文工作并提出后续改进方向。

2 概述

2.1 系统建设目标概述

本文系统面向中小学生编程学习与教师教学管理两个侧面展开。**学生**侧重点在于课程学习、在线练习、报错理解和智能提示；教师侧重点在于课程资源维护、学习过程观察、消息互动和必要干预。本设计不把大模型作为独立功能入口，而是将其嵌入代码解释、报错解析、问题分解、抽象建模和算法设计引导等学习环节中。

从工程边界看，系统需要同时完成前端交互、后端接口、数据持久化、代码安全执行和模型服务调用。各部分既要能够独立维护，也要在学习流程中形成完整衔接。后续章节将围绕这一目标，依次说明需求、设计、实现和测试过程。

2.2 大语言模型与智能辅学

大语言模型在语义理解与文本生成上表现日趋稳定，落在教育领域，已自然渗透到问答、概念解释、纠错和学习引导等环节^[2-3]。在编程教学里，这类模型能做两件事：一是回应编程疑问，二是把抽象概念切换到初学者能听懂的表达，让专业术语和晦涩描述不再挡路。从现有文献看，LLM 在编程教育中的几个典型切入口包括代码解读、排错支持、形成性反馈、自动评测和知识建模^[3,6]。

LLM 的教学价值并不等同于直接替代教师。相关研究表明，模型在真实性、稳定性、可追责性与教学适配性方面仍存在明显边界^[10-12]。尤其在编程学习场景中，模型可能生成表面连贯但实质有误的解释，也可能过早给出完整答案，削弱学生自行分析、调试和抽象概括的过程^[10,9]。如果缺少明确教学约束，LLM 容易从“辅助学习工具”转向“替代思考工具”。

基于上述认识，本文在平台设计中未将 LLM 设定为自动判分器或唯一答案源，而是将其定位为过程性辅学模块。其作用主要体现在三个位置：学生卡住时提供适量提示，错误定位时给出结构化解释，问题分解时提供思路引导。同时，教师管理端保留审阅、追踪与干预空间^[8,5]。这种定位更符合中小学生编程学习的认知发展特点，也有利于维持课堂教学中的教师主导性。

2.3 前端开发技术

前端一侧，Nuxt 3 担纲主框架，Vue 3 的组合式 API 驱动页面逻辑，Tailwind CSS 搭 DaisyUI 处理样式，TypeScript 管好类型安全。这套组合对交互型页面的开发需求应

付得过来，也顺手把前端拆成了页面与组件的两级结构。

2.4 后端开发技术

后端以 FastAPI 作为核心 Web 框架，用于组织业务接口、请求校验和接口文档。数据库访问通过 SQLAlchemy 实现 ORM 映射，业务认证采用 JWT 机制完成身份校验。Redis 承担部分缓存处理，以降低高频场景下的响应压力。

2.5 数据存储与资源管理技术

数据层面，结构化数据主要存储在 SQL Server 中，包括用户、课程、任务、知识点、资源、评论消息、点赞关系与学习记录等信息。教学视频等非结构化资源结合阿里云 OSS 进行对象存储，避免把大文件直接放入业务数据库，也便于后续资源访问与管理。

2.6 在线编程与安全执行技术

在线编程是本平台的核心功能之一。为降低服务器端代码执行风险，平台将 Python 代码运行迁移到浏览器端完成：前端页面通过 Web Worker 承载独立执行线程，Worker 加载 Pyodide 运行时，并依托 WebAssembly 在浏览器沙箱内执行 Python 代码。运行过程中的标准输出、错误信息和输入等待状态通过消息机制回传页面，后端主要保留课程、任务、记录和资源等业务接口职责。

2.7 系统需求分析

2.7.1 应用场景分析

本项目瞄准的是中小學生编程学习和教师教学管理两条线。典型场景不止一种：课内教学、课后练习、在线答疑、资源分享和学习跟踪都在覆盖范围内。

落到真实节奏里看，平台的使用不压在单个时点，而是顺着“课前准备 → 课中讲解 → 课后巩固 → 过程评价”一路铺下来。课前，教师整理课程资源、配好任务要求、挂上知识点；课中，学生窝在一个环境里写代码、跑调试、随时问；课后，教师翻学习记录看常见错误和完成进度，再调下一次课。这里平台的角色不是单纯代码运行器，而是一个串起教学流程的学习支架。

在学生侧，本平台尤其适用于编程入门阶段的高频卡点场景。例如，学生面对报错信息时，往往能够看见结果，却难以理解错误含义，更难判断修改顺序；面对综合任务时，也常出现“知道题目要做什么，但不知道第一步从哪里开始”的情况。这两类问题是传统编程教学中容易造成挫败感的环节，也是引入智能辅学能力较有现实价值的环节。

在教师侧，平台更强调过程管理而非结果收集。与只看最终提交结果的方式相比，教师更需要知道学生在哪个知识点反复出错、在哪个阶段停留时间过长，以及哪些资源真正被学生使用。基于此，平台在设计时不仅关注作业提交与课程展示，也重视消息互动、行为记录和错误追踪等过程性数据。

2.7.2 用户角色分析

两类用户构成了本平台的核心使用群体。

- (1) 学生用户：完成课程学习、代码编写、任务提交、错误分析、消息互动和学习记录查看。
- (2) 教师用户：查看学生学习进度、统计错误信息、管理教学资源、发布或维护课程任务并进行师生互动。

上述两类角色在平台中的关注点并不相同。学生用户更关心操作流程是否清晰、反馈是否及时、解释是否易懂，因而其需求主要集中在界面易用性、反馈即时性和辅学内容的可理解性方面。尤其对于编程基础较弱的学习者而言，若系统返回的提示过于抽象，或一次给出过多信息，反而会增加理解负担^[13]。

教师用户则更重视平台的组织能力和可管理性。教师不仅需要发布课程、维护知识点和上传资源，还需要借助平台掌握学生学习动态，并对常见问题进行集中处理。因此，教师端不应只是学生端功能的简单镜像，而应当具备更强的管理属性、统计属性和教学干预属性。

从权限控制角度看，学生与教师的角色边界也必须明确。学生侧应以学习和提交为主，教师侧则拥有课程维护、资源管理、学习记录查看等更高权限。只有在角色职责清晰的前提下，平台才能在保证数据安全的同时维持较好的交互秩序。

2.7.3 功能需求分析

学生端功能需求

学生端应满足以下功能需求。

- (1) 登录认证与身份识别。
- (2) 查看课程与学习任务。
- (3) 在线编写并运行代码。
- (4) 获取代码解释与报错分析结果。
- (5) 使用智能问答功能寻求帮助。
- (6) 与教师或同学进行消息互动。

(7) 查看学习进度、错误记录和推荐资源。

上述功能需求可以归纳为四个层面：学什么、怎么练、卡住时怎么办、学完后怎么看。学生需要通过课程页了解当前学习内容，也需要借助知识点结构明确任务边界。只有知道自己正在学习哪部分内容、对应哪些具体任务，学生才能进一步判断应达到的学习目标。如果课程结构展示不够清晰，学生在任务开始前就可能产生方向性困惑。

在练习过程中，学生需要稳定的在线编程环境。该环境不仅要支持代码输入与运行，还要保证输出结果和错误信息能够及时返回。对于初学者而言，代码运行反馈越延迟，学习链条就越容易中断。因此，在线编程能力不是平台的附属模块，而是学生端体验的核心支点。

当学生遇到理解障碍、运行错误或任务难以拆解时，平台应当提供适量而非过量的辅助。学生端智能辅学功能的关键不在于“回答得多完整”，而在于“回答能否让学生接得住”。这意味着系统在设计时应优先支持解释、提示和引导，避免直接输出大段答案。

学生还需要能够回看自己的学习过程。学习进度、错误记录和历史互动内容不仅有助于学生形成反思，也能减少重复性提问，提高后续学习效率。对于编程初学者而言，看见自己从频繁报错到逐步修正的过程，本身就是一种可感知的成长反馈。

教师端功能需求

教师端应满足以下功能需求。

(1) 查看班级学生学习进度。

(2) 统计学生错误类型与学习表现。

(3) 管理教学资源。

(4) 维护课程任务与知识点内容。

(5) 与学生进行消息交流。

教师端的功能需求可概括为“可组织、可观察、可干预”三个方面。教师应能够围绕课程主题建立较清晰的内容结构，包括课程分类、知识点关联、任务配置与资源上传，使平台内容与实际教学进度保持一致。

教师还应能够方便地查看学生在学习过程中的关键状态信息，例如任务完成进度、错误分布、活跃情况和典型问题。若教师只能看到最终提交结果而无法掌握过程数据，平台对教学改进的支撑作用就会被大幅削弱。

当教师发现学生存在集中性问题时，应能够通过消息互动、资源补充或任务调整等方式及时介入。面向中小学生的编程学习平台不能只强调学生自学，还必须保留教师干预通道。这样，平台中的智能功能才不会与课堂教学脱节，而会成为教师教学能力的延伸。

智能辅学功能需求

智能辅学需要实现面向中小学生的以下能力。

(1) 编程相关自由问答。

(2) 代码整体与逐行解释。

(3) 错误信息分析与定位。

(4) 编程任务问题分解。

(5) 抽象建模引导。

(6) 算法设计流程引导。

与传统问答系统相比，本平台的智能辅学需求具有更强的教学场景约束。回答对象是中小学生，输出内容必须兼顾正确性、可理解性与适龄性；问题上下文往往与课程任务、学生代码和报错结果绑定，平台不能脱离具体学习情境空泛作答；同时需要避免学生对模型产生过度依赖，因此在部分场景下应优先给出分步提示而非终局答案^[10,9]。

围绕上述教学约束，智能辅学功能至少应满足三项要求。模块需要识别问题类型，并根据不同任务调用不同的回答模板；同时能够结合学生输入内容生成有针对性的反馈，而不是返回通用化说明；此外还需要对输出进行必要约束，使结果既服务学习，又不突破课堂任务边界。这些要求共同决定了智能辅学模块不能只停留在接口接入层面，而必须在系统设计层面进行专门建模。

2.7.4 非功能需求分析

性能需求

性能方面，需具备稳定的接口响应能力和并发处理能力，能够满足日常教学场景中的多用户访问需求。

考虑到平台的主要使用场景集中在课堂或作业提交时段，系统在性能上不仅要管得住普通页面访问，还要扛得住高频操作的扎堆涌入。比如多名学生同时打开课程页、运行代码或请求报错解析，都会对接口响应和任务处理提出更高要求。因此性能需求的发力点应侧重课程访问、代码运行和智能问答三条高频路径，而不是把精力平均摊给所有模块。

安全需求

安全方面，需保证用户身份认证安全、代码执行安全和教学数据安全，避免恶意访问、非法调用和危险代码执行。

其中，代码执行安全是本平台区别于一般教学信息系统的重要非功能需求。由于平台允许用户提交并运行代码，因此系统必须在执行前进行语法检查、危险调用过滤和超时控制，并在执行后隔离结果回传范围。也就是说，在线编程功能既要允许学生完成必要练习，又要通过边界限制和防护机制降低执行风险。

可扩展性需求

扩展方面，应为后续接入更多课程资源、更丰富的大模型能力以及知识库检索增强等功能保留扩展空间。

从平台演进角度看，可扩展性体现在不同层面。业务模块应能够继续增加新的课程类型、任务类型和行为记录类型，而不破坏既有结构；智能辅学模块应能够替换或增加新的模型服务，而不影响整体调用流程；平台还应能够为后续接入知识库检索增强、学习者画像和个性化推荐等功能预留接口与数据基础。

易用性需求

由于平台面向中小學生，系统界面和交互流程应尽量简洁直观，智能反馈内容也应贴近学生认知水平。

易用性需求不仅体现为页面是否美观，更体现为操作是否顺手、提示是否明确以及反馈是否容易理解。对于中小學生而言，过多的操作层级、过长的文字说明和过于专业化的表达都会直接提高使用门槛。因此，本平台在交互设计上应尽量缩短核心路径，在反馈设计上减少术语堆叠，使学生把主要注意力放在学习任务本身，而不是放在理解系统如何使用上。

2.7.5 可行性分析

技术可行性

当前前后端框架、大模型接口能力和数据库技术较为成熟，能够支撑本系统实现。

从开发实现角度看，Nuxt 3、FastAPI、SQLAlchemy、SQL Server 与 Redis 均属于技术体系较成熟、文档较完善的技术方案，能够覆盖页面交互、接口开发、数据持久化与缓存优化等主要需求。同时，大模型接口的接入方式已经标准化，便于将自然语言处理能力融入既有业务流程。因此，本文所提出的平台方案在技术栈组合上不存在明显不可实现障碍。

经济可行性

本项目使用的主要开发工具与框架具有较好的可获得性，部署成本相对可控，具备开发基础。

此外，平台的基础模块主要依托通用开发框架实现，软件层面的初始建设成本相对可控。虽然大模型调用会带来一定服务成本，但通过场景限制、请求约束和缓存设计，可以控制无效调用和重复调用。对于毕业设计层面的系统实现而言，现有资源条件能够支撑平台原型的开发、联调与展示。

运行可行性

功能上与教学流程匹配度较高，实际使用中，学生与教师均可在已有使用习惯基础上完成操作，具备实际运行可行性。

从使用方式看，与学校日常教学流程具有较高一致性。教师侧的课程发布、任务布置和资源分享，本身就是教学中的常规动作；学生侧的学习、练习、提问和查看反馈，也与日常编程学习行为高度重合。也就是说，本平台并未试图重造一套脱离教学实际的流程，而是在原有教学链条上增加数字化支持，因此具备稳定的运行基础。

3 系统总体设计

3.1 系统设计目标

总体设计以模块化、可扩展、安全可控为三条基线。**本项目**既要把课程学习、在线编程和师生互动这些基础教学流程跑通，也要在这些流程里接上大模型反馈机制——不是为了多塞一个智能问答入口，而是要在学生卡壳、代码报错和任务拆解等场景中，给出及时且可控的支持。

3.2 系统总体架构设计

整体采用前后端分离架构。前端负责页面展示与交互逻辑；后端承担业务处理，并完成认证鉴权、代码执行、数据持久化与智能能力调度。总体架构如图 3-1 所示。

在该架构中，前端通过统一接口调用后端服务。后端按业务域对请求进行分发，覆盖用户管理、课程管理、任务管理、消息互动、资源管理、行为记录与智能辅学等模块。智能辅学模块内部并不采用单一路径：涉及自然语言与代码解释的请求通过大模型接口处理；在线代码运行与错误返回则由安全代码执行器完成。

3.3 系统功能模块设计

根据业务需求，平台功能划分为课程学习、在线编程、智能辅学、资源管理、消息互动、学习记录和系统管理等模块，如图 3-2 所示。各模块围绕学生学习流程和教师管理流程展开：学生端侧重学习、练习与反馈，教师端侧重资源维护、过程观察与必要干预。

3.4 前后端结构设计

3.4.1 前端结构设计

前端结构采用“页面 + 组件”的方式组织。页面层覆盖登录页、学生主页、教师主页等入口页面，并包含课程更新页、事件记录页与事件追踪页等业务页面。组件层积累可复用单元，例如课程分类选择器、课程搜索选择器、知识点选择器，以及聊天组件和富文本编辑器等。

在前端结构组织上，本文采用“页面承载业务、组件复用交互”的设计思路。页面层以流程编排为主，负责触发数据请求；组件层聚焦交互单元复用，并管理局部状态。这

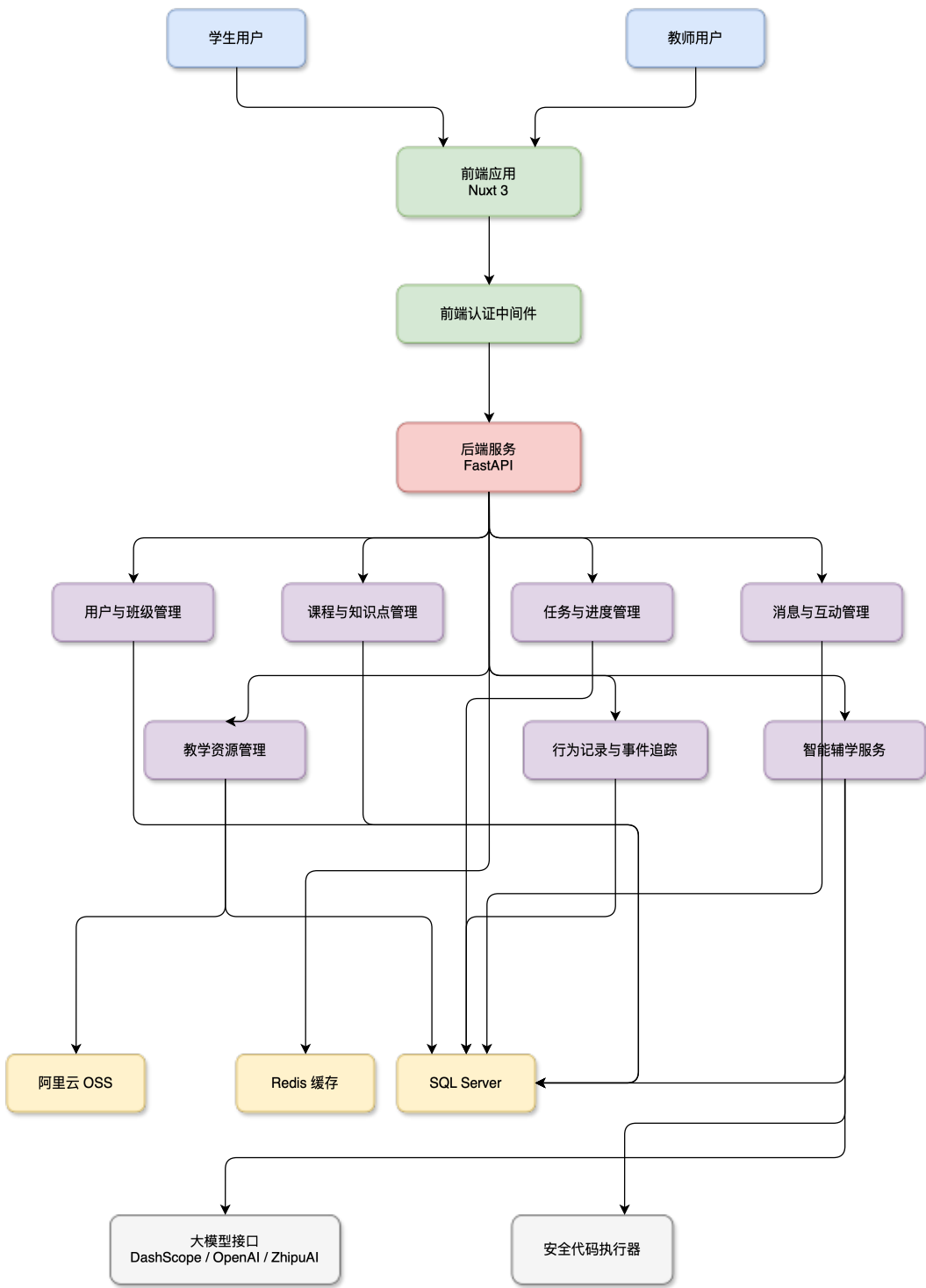


图 3-1 系统总体架构图

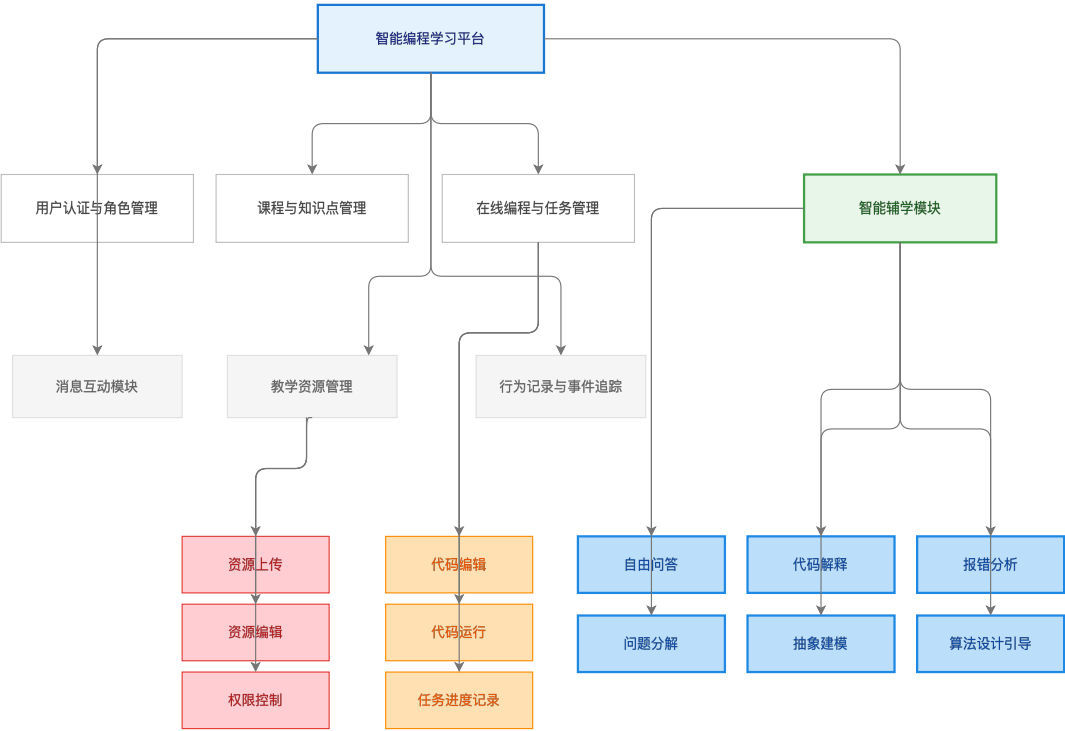


图 3-2 系统功能模块图

样划分后，不同业务页面的职责边界更容易保持清晰；在课程筛选、聊天交互与资源编辑等高频场景中，也能减少重复开发工作。

学生端页面更关注学习路径长度，目标是减少学生完成“查看课程、进入任务、编写代码、查看反馈”主流程时的页面跳转与操作负担。教师端页面则把重点放在内容维护效率与信息聚合能力上，课程编辑、任务配置、资源管理和学习记录查看等常用入口尽量集中呈现。

3.4.2 后端结构设计

后端按路由模块编排业务接口，认证依赖统一注入，一刀切权限。公开接口只管登录和基础班级访问，剩下的——课程、任务、记录、资源、消息、点赞——统统在鉴权之后才放行。这么组织，路由职责不糊，权限边界不混，后面加新模块也顺手。

在后端结构上，系统按业务域划分接口模块，使用户管理、课程管理、任务管理、资源管理、消息互动和学习记录等功能在相对独立的边界内演进。与按页面或按角色分散定义接口相比，这种方式更符合服务端对数据与业务规则统一管理的需求；后续增加新模块时，也更有利于保持接口体系的连续性。

除此之外，后端结构还需兼顾两件事：通用能力要集中归纳，业务能力要各自解耦。认证校验、异常处理、统一返回格式等能力抽成公共机制，不让每个接口都重写一遍；课程维护、代码运行和智能辅学这类业务逻辑则分模块安置，彼此之间松耦合。这样处理，整体一致性站得住，业务扩展也留有余地。

4 系统详细设计

4.1 数据库设计

4.1.1 数据库概念结构设计

数据库围绕用户、课程、任务、知识点、资源、评论消息、点赞关系和学习记录等核心实体展开。主要关系如图 4-1 所示。

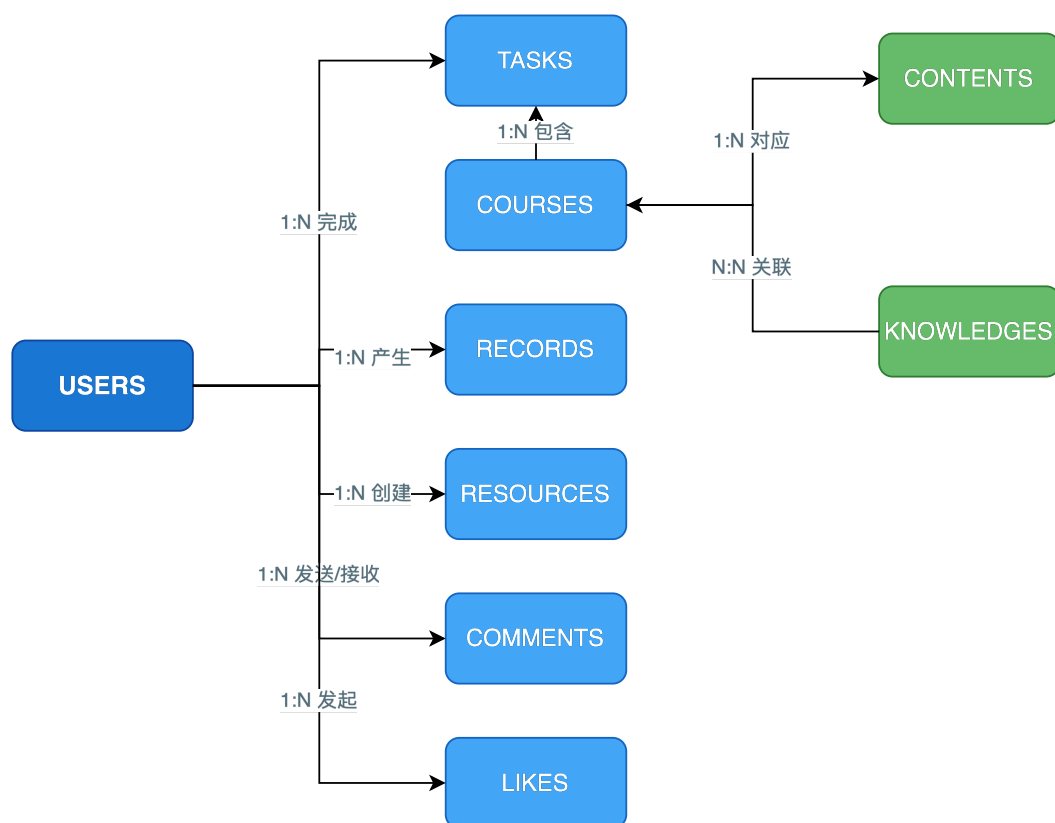


图 4-1 数据库概念结构图

从概念设计角度看，数据库不单为页面留出存储格子，更关键的是把平台内的业务关系表达清楚。用户实体串起学习行为和互动行为，课程实体串起知识点、任务和资源，记录实体则留住学习过程中的关键足迹。这么一来，数据库存的就不只是结果快照，也把过程痕迹保存了下来，给后面的统计分析和教学干预打好底子。

在概念模型的基础上，本文进一步细化各实体的属性与关联约束，得到如图 4-2 所示的系统数据库逻辑结构图（E-R 图）。该图描述了用户、课程、任务、知识点、资源、评论、点赞和学习记录等实体之间的为主外键关系及业务关联，为后续主要数据表设计提

供依据。

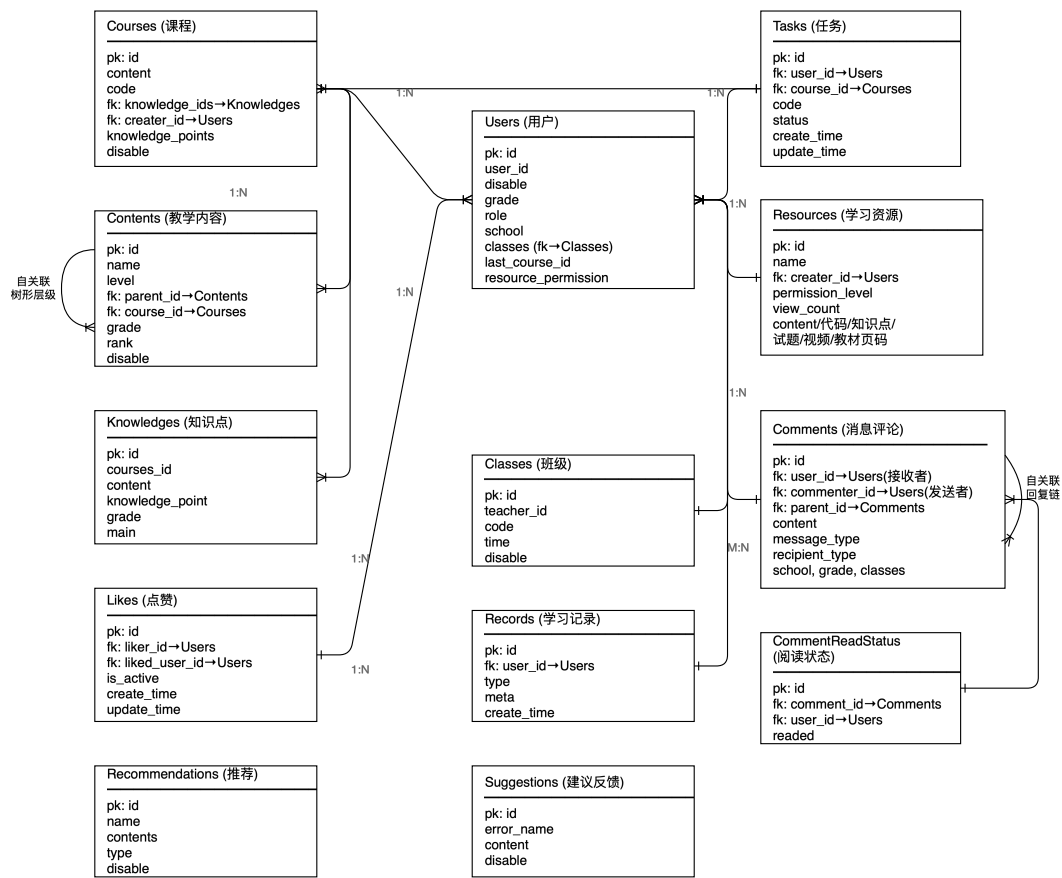


图 4-2 系统数据库逻辑结构图（E-R 图）

4.1.2 主要数据表设计

主要数据表设计如表 4-1所示。

落到逻辑设计层面，users 表管权限识别与角色划分；courses、contents 与 knowledges 三表协同组织课程内容；tasks 表存学生的完成状态和代码提交结果，resources 表管教学材料。comments 与 likes 负责互动关系，records 表记下页面访问、代码运行和错误类型等过程数据。这些表通过主外键或业务关联字段串在一起，让课程学习、任务执行和行为分析三条主线都有处落脚。

数据库设计的重点不在表数量本身，而在业务语义是否清楚。若单表承担过多职责，后续维护中容易出现字段膨胀和语义混杂；若表之间关联过松，数据又难以被有效利用。因此，本文在设计时尽量让每张表对应相对明确的业务职责，并为后续扩展预留适当空间。

表 4-1 主要数据表设计

数据表	说明
users	存储用户基础信息、角色、学校、班级等
courses	存储课程信息和课程内容索引
contents	存储课程层级内容结构
tasks	存储学生任务代码、状态与时间信息
knowledges	存储知识点与内容关联信息
resources	存储教学资源与权限信息
comments	存储消息与评论内容
likes	存储点赞关系
records	存储学习行为和错误记录

4.2 智能辅学模块设计

智能辅学模块的处理流程包括“提问输入”“场景识别”“提示词构造”“模型生成”和“结果返回”五个环节。面对不同学习任务，系统分别调用自由问答、代码解释、报错分析、问题分解、抽象建模和算法设计等功能接口；其处理流程如图 4-3 所示。

为避免大模型输出与教学目标脱节，本文进一步将该模块细化为“场景识别—约束生成—教学化输出—风险兜底”的四层结构。

在场景识别层，系统不把学生输入一律视为普通问答，而是结合问题文本、代码内容、运行结果和任务上下文划分请求类型。自由问答、代码解释、报错解析、问题分解、抽象建模和算法设计引导六类任务，会进入不同的提示模板与输出结构，避免模型在所有场景下使用同一种回答方式。

在约束生成层，系统将学生年级、课程主题、知识点标签、任务目标和回答边界写入提示内容^[5,14]。其中，教学约束要求回答循序渐进、语言适龄，并避免无前提地直接给出完整答案；安全约束则要求拒绝危险代码、规避越权请求，并限制与课程目标无关的输出扩散。这一思路与现有研究中关于人在回路和教学目标匹配的观点一致^[8,10]。

在教学化输出层，系统并不追求一次性给出最完整答案，而是优先保证回答的教学可用性。以报错解析为例，系统要求模型依次说明错误含义、错误位置、可能原因与修改建议；在算法设计引导场景中，则先帮助学生明确输入、输出与关键步骤，再逐步组织流程。通过这种结构化输出，平台更容易把模型回答转化为学生能够理解和消化的学习材料。

在风险兜底层，系统保留日志记录、人工复核与规则拦截机制。若出现高风险输出、低置信度回答或涉及完整作业答案的请求，平台会优先采用弱化回答、返回思路框架或提示教师介入等处理方式。该层并非附加功能，而是教育场景接入 LLM 的必要条件；缺少兜底机制的模型系统，如同制动不足的车辆，能力越强，进入课堂后的不可控性也越需要被约束。

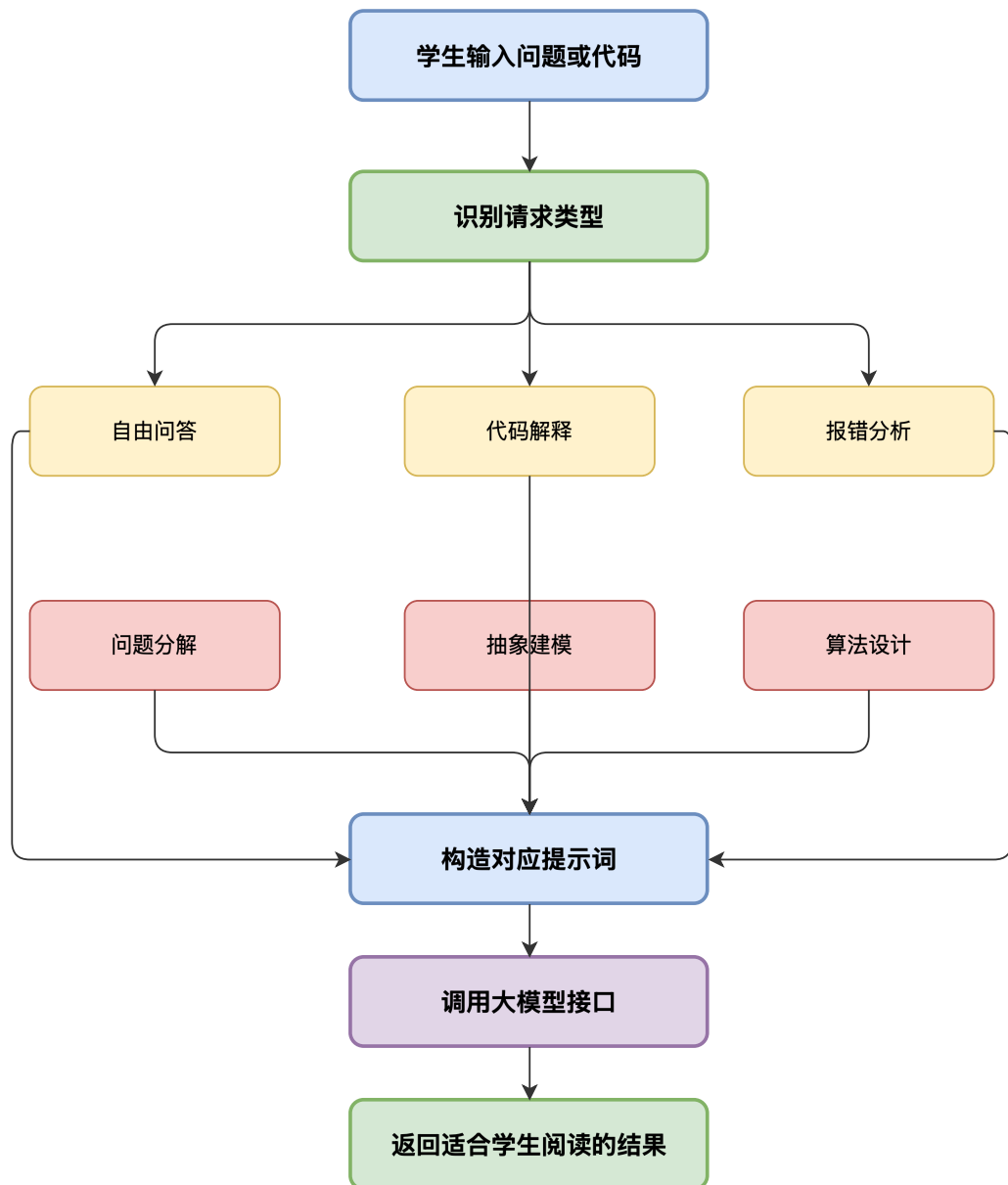


图 4-3 智能辅学模块处理流程图

4.3 系统部署设计

前端通过 Node 环境部署 Nuxt 应用，后端侧使用 Uvicorn 运行 FastAPI 服务。数据库采用 SQL Server，缓存采用 Redis；静态资源与教学视频存储于阿里云 OSS。若后续面向更大规模课堂使用，可在该基础上接入负载均衡和弹性伸缩机制。

从部署关系看，前端应用主责页面渲染和交互请求，后端服务专管接口调度。数据库扛结构化数据的持久化，Redis 负责部分热点缓存，OSS 则托管教学视频一类的非结构化资源。各组件的职责边界划得清楚之后，访问压力自然分布开，不至于全部堆到单一服务节点上。

在毕业设计的实现规模下，上述部署方式能够满足系统演示和功能联调需求；若面向更大规模的教学使用，则还需进一步考虑服务容错、日志监控、备份恢复和并发调度等问题。部署设计类似教学楼中的管线系统，平时不直接呈现在用户界面上，但一旦规划混乱，后续功能运行都会受到牵连。论文中明确部署关系，有助于说明系统从设计到运行的落地路径。

5 软件实现

5.1 用户认证与权限控制实现

用户认证基于 JWT 实现。登录成功后，前端将必要的身份状态持久化，并借助全局认证中间件拦截需登录权限的页面访问；后端侧通过依赖注入完成身份校验，并将公开接口与受保护接口分组管理。

认证模块的核心目标在于保证“用户身份可信”，并让“接口访问有边界”。学生与教师共用同一平台入口，但两类角色可访问页面与可执行操作并不相同。为此，登录流程不仅判断认证状态，还会读取角色属性，再据此决定可见菜单、可访问路由，以及可调用接口范围。

前端侧主要负责维持登录态，并执行页面级拦截。用户完成登录后，系统保存必要的身份状态；页面切换时，前端先判断当前页面是否需要认证，从而提前挡住未授权访问，减少无效请求进入后端。后端侧承担最终校验职责，通过统一认证依赖校验令牌有效性与用户身份，并在关键接口中，进一步核对角色权限。

权限控制模块关注的不仅是“谁能进来”，更关心“进来之后能做什么”。如果缺少细粒度权限边界，教师端的资源维护能力与学生端的学习操作就可能相互混淆，进而影响数据安全与系统秩序。认证与权限控制虽在界面上并不显眼，却是平台稳定运行的基础。

5.2 课程与知识点管理功能实现

课程内容通过课程分类选择器、课程搜索选择器与知识点选择器进行组织。在课程更新页面，内容编辑、知识点关联与代码内容维护均可完成；前端借助富文本编辑器提升录入效率，后端负责课程与知识点数据的持久化。

课程与知识点管理模块的实现重点是内容结构化。若课程内容仅以零散条目呈现，学生学习时很难形成清晰的知识路径^[15]，教师维护时也不易保证内容一致性。基于此，平台在课程、知识点与资源之间建立关联关系，使课程不只用于展示，还承担知识组织与任务承载的基本职责。

在教师使用流程中，课程维护通常包含新增课程、编辑课程描述、关联知识点、补充示例代码以及调整内容顺序等操作。为降低维护成本，前端页面尽量将这些动作收拢到同一工作界面，减少教师在多个页面间切换。后端接收结构化表单数据，并保证课程与知识点的关联关系能够稳定写入数据库。

对学生而言，该模块的价值主要体现在“看得懂课程结构”以及“找得到对应内容”。课程模块设计是否合理，会直接影响后续任务管理、智能问答与资源推荐等模块的使用

体验。课程结构清晰，后续功能才有稳定落点；结构若混乱，即使其他模块功能较完整，也容易被用成零散工具。

5.3 在线编程与代码运行功能实现

在线编程功能是系统核心模块之一。学生在课程页面编写 Python 代码后，前端不再将代码交由后端进程直接执行，而是在浏览器端创建 Web Worker，并在 Worker 内加载 Pyodide 与 WebAssembly 运行环境。后端运行时接口只负责分发 Pyodide、Worker 脚本及相关静态资源，代码执行、输出捕获和错误回传均在浏览器沙箱内完成。浏览器端代码执行架构如图 5-1 所示。

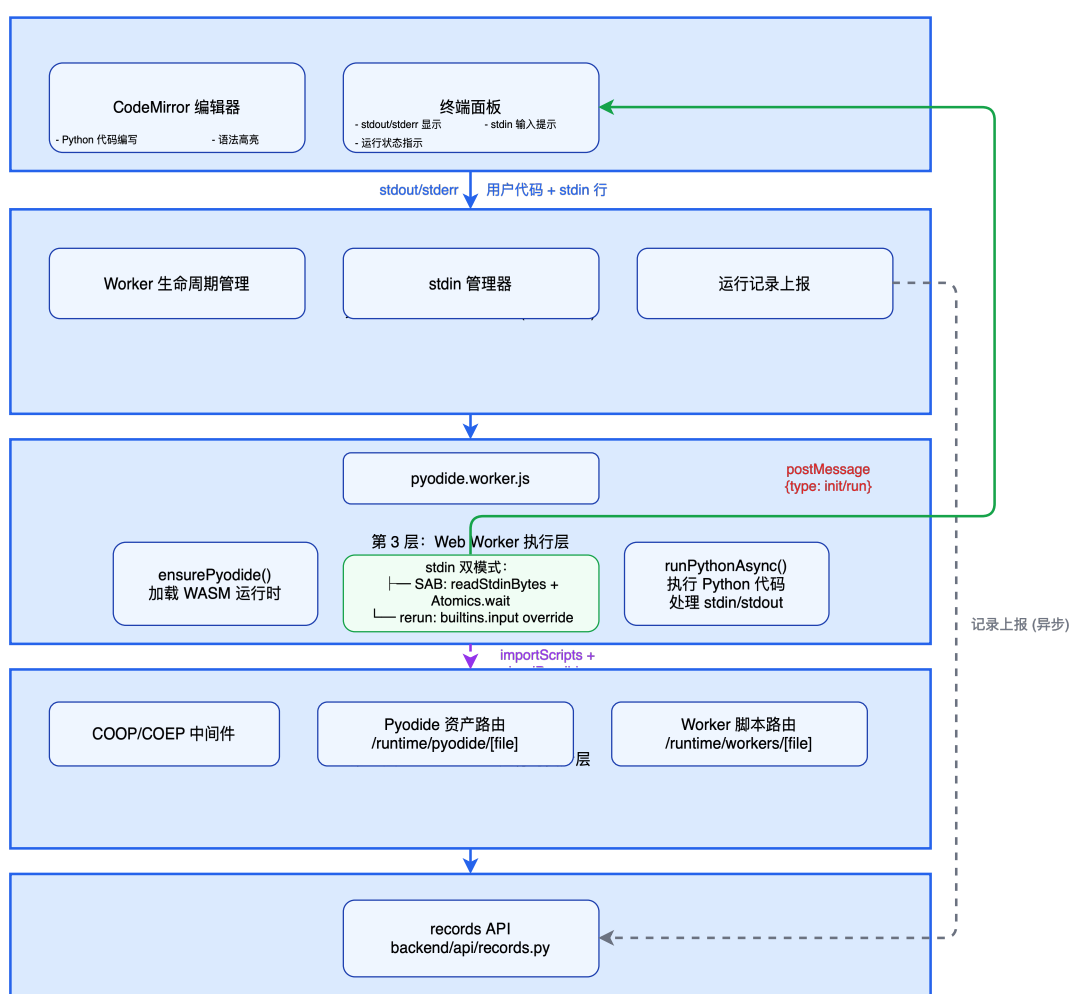


图 5-1 浏览器端代码执行架构图

从实现逻辑看，在线编程与运行流程大致经历五步：终端初始化、运行时加载、代码投递、浏览器端执行与结果回传。页面加载后初始化 Worker，并请求 Pyodide 入口脚本、标准库压缩包和 Wasm 文件；学生点击运行后，页面通过 `postMessage` 将代码与标准输入队列发送给 Worker；Worker 调用 Pyodide 的 Python 执行接口运行代码，并

将标准输出、标准错误、输入等待和运行结束状态持续回传页面。该模块的主要执行流程如图 5-2 所示。

其中，Worker 隔离与 Pyodide 运行时加载属于关键环节。代码若在后端服务进程中直接执行，平台稳定性和服务器安全性都会承受较大压力；迁移到浏览器沙箱后，学生代码主要占用本地浏览器资源，服务端暴露面明显缩小。同时，Worker 与主页面之间只通过消息传递交换状态，页面线程不会被 Python 执行过程直接阻塞，终端区域也能持续接收输出、错误和输入请求。

代码运行模块同时承担行为记录功能。每一次运行请求无论成功与否，都可能折射学生的学习状态：同类语法错误反复出现，往往意味着相关知识点尚未真正掌握；某一任务若长时间停留在运行失败，也可能说明，学生在任务分解阶段已经遇到困难。因此，前端在获得 Pyodide 执行结果后，仍会把代码、输入、输出、错误类型和运行时标识写入学习记录接口。代码运行结果既服务于当前页面反馈，也为教师后续分析学生学习过程提供数据基础。

5.4 大模型辅助教学功能实现

5.4.1 自由问答功能实现

面向编程相关问题，本平台提供流式问答能力。为限制回答范围并贴合学生使用场景，后端在请求发送给模型前加入系统提示词，用于约束回答主题与回复方式。

在该功能中，系统不把模型简单视为开放聊天接口，而是通过提示约束将其限定在编程学习支持场景内。模型回答的重点放在概念解释、思路提示与学习引导上，减少与课程无关的泛化内容，以降低偏离教学目标的概率。

进一步地，相关研究表明，结构化且可回合追踪的同伴对话反馈机制有助于促进深度学习过程中的反思与策略修正^[16-17]。据此，本文在自由问答实现中强调问题澄清、分步追问与关键点回收，尽量避免一次性长答案挤占学生的自主思考空间。

5.4.2 代码解释功能实现

代码解释功能以学生提交的代码为输入：模型先概括程序整体功能，再逐步解释关键语句，帮助学生形成从整体到局部的理解路径。

本文在该功能设计中更强调解释顺序，而不是单纯增加解释长度。对于初学者而言，若系统一开始就逐行展开所有细节，学生往往会陷入局部信息而失去整体把握。基于这一点，本平台优先要求模型先说明程序“在做什么”，再说明“为什么这样做”，随后，进入关键语句解释。这种组织方式更符合中小学生对整体到局部的理解路径。

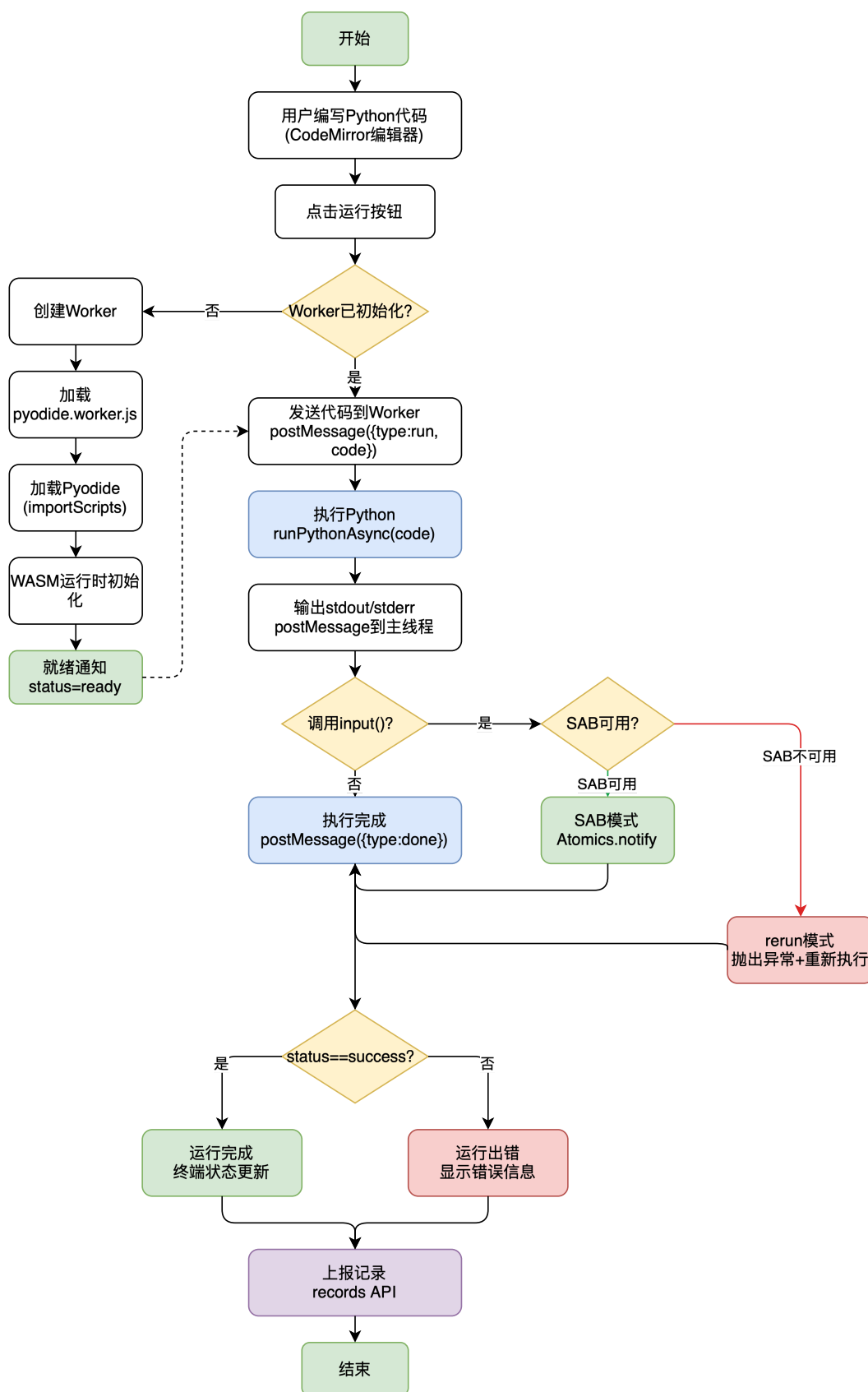


图 5-2 浏览器端代码执行流程图

5.4.3 报错解析功能实现

报错解析功能会同时接收学生代码与运行输出，并通过提示词约束模型采用“三步式”输出：先解释错误含义，再定位错误位置，最后给出启发式修改引导。这种组织方式更适合中小学生学习中的循序渐进式反馈。

与直接展示原始报错信息相比，报错解析功能更强调“翻译”与“拆解”。不少初学者并不是完全不能改错，而是读不懂错误信息，也说不清应先看哪一行、先改哪一步。因此，系统在此处引入结构化输出，并不是为了增加模型参与感，而是为了把原本碎片化、技术化的报错信息转写为学生能够操作的修改线索。

5.4.4 问题分解功能实现

基于课程内容与学生任务描述，复杂编程任务被拆分为若干可执行步骤，以降低问题理解难度。

问题分解功能主要面向综合任务场景。学生面对较长任务描述时，常会出现“不知道从哪里开始”的情况。对此，系统不直接替学生完成程序，而是将任务拆分为输入分析、数据处理、条件判断、循环组织与结果输出等步骤，让学生先找到入口，再推进后续实现。

5.4.5 抽象建模功能实现

抽象建模功能用于引导学生从具体问题中提炼一般规律，帮助其建立更稳定的问题求解方式。

这一功能的重点不在于给出标准模型，而在于帮助学生完成“从具体到一般”的过渡。中小学生学习编程中常见的情况是：在单个例题里能够模仿做法，但很难把方法迁移到相似问题。抽象建模功能的价值正在于提炼输入对象、处理规则与输出目标，帮助学生逐步形成可迁移的思维框架。

5.4.6 算法设计引导功能实现

算法设计引导功能通过“开始、输入、处理、循环、输出、结束”等模板，提示学生组织思路，帮助其逐步形成结构化的算法表达能力。

在实现上，该功能更接有没有系统功能页面截图？

要截图介绍近思路支架，而非答案生成器。系统用固定过程模板帮助学生组织解题步骤：先明确流程结构，再进入代码表达阶段。这样处理后，原本容易混乱的解题过程被拆成可检查的步骤链条，学生在编码前的思路负担也随之降低。

5.5 师生互动与消息功能实现

师生之间以及学生之间的互动由评论消息相关接口实现。系统支持消息发送、消息读取、未读状态管理和对话内容展示，可用于课后答疑与学习交流。

消息模块的实现意义在于补足教学平台中的“人际反馈通道”^[16-17]。如果平台只有课程、任务和模型回答，而缺少教师与学生之间的直接互动，许多需要情境判断和个别化解释的问题仍然难以得到有效处理。消息模块并不是对智能功能的重复建设，而是对其必要补充。

在具体实现中，系统需要同时处理消息内容、发送关系、接收状态与未读标记等信息，使互动过程既可追踪又便于管理^[17]。对教师而言，消息模块能够承载课后答疑与个别提醒；对学生而言，则能够在模型回答不足或需要确认任务要求时，保留向教师继续求助的正式通道。

5.6 教学资源管理功能实现

教学资源的创建、修改、删除与权限设置功能集中在教师端。资源信息包括名称、内容、代码、教材页码、视频链接和知识点标签等，用于支撑课堂教学与课后扩展学习。

教学资源管理模块的重点在于资源与教学内容之间的可关联性。若资源只以文件形式独立存在，其教学价值往往取决于教师额外解释；若资源能够与课程、知识点和任务建立对应关系，学生在学习过程中，便更容易找到“当前问题对应的辅助材料”。因此，本平台中的资源管理不是简单文件上传，而是面向教学组织的资源编排。

资源权限的控制在实际使用中同样有价值。有些资源面向全体学生开放即可，有些则更适合留在教师手里，作为备课参考或分层教学的补充。一套权限机制管下来，同一批资源既敞开了公开学习通道，也护住了教师内部管理需求，复用效率自然上去。

5.7 学习行为记录与事件追踪功能实现

用户学习过程中的页面访问、代码运行、错误类型等行为数据会被记录，并通过可视化页面，展示学习记录与互动情况，为研判学生学习进度与错误分布提供数据依据。

从平台价值的视角看，学习行为记录模块扮演着系统“观察窗”的角色。没有它，平台一样能跑教学功能，却始终答不了这一问：学生究竟是怎么用这些功能的？把页面访问、代码运行、错误类型和交互过程记下来，原本看不见的学习轨迹就能转化成可分析的数据线索。

学习行为记录模块为后续教学干预提供依据，而非仅用于统计汇总。借助行为数据，高频错误知识点与偏难任务均可被定位，长期缺乏有效互动的学生也能被发现。对毕业设计而言，该模块也为后续引入学习者建模、分层推荐与教学效果评价提供了基础数据入口^[1]。

5.8 系统界面展示

本节通过系统运行截图，对平台主要用户界面进行展示与说明。

5.8.1 登录界面

图 5-3展示了系统的登录界面。整体采用居中卡片式布局，顶部“请登录”标题简明扼要。表单区域包含账号输入框与课程码输入框，其中课程码字段在输入时以密码掩码显示，这一设计考虑了课堂环境中多人在场的使用场景，可避免邻座学生窥视。蓝色主色调的登录按钮负责触发 JWT 令牌获取与前端状态持久化逻辑。对于尚未创建班级的教师用户，界面还提供绿色“生成课程码”按钮，点击后调用后端接口生成唯一课程码并回显，同时支持一键复制，便于教师快速分发给学生。登录成功后，系统读取用户角色字段并自动路由至对应主页：学生进入资源页，教师进入管理页。

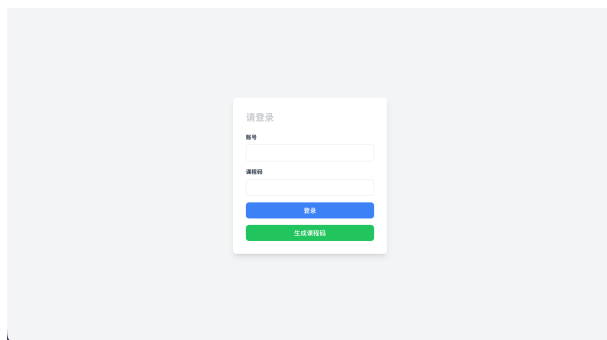


图 5-3 系统登录界面

5.8.2 学生资源页仪表板

学生端主界面采用 2×2 网格布局，四个功能卡片对核心信息进行分区展示，如图 5-4所示。左上角为课程进度表，全班完成率按列展示，学生可直看到自身进度；右上角展示错误类型分布，系统依据代码运行记录统计错误分布，并以水平进度条呈现高频错误的集中情况。

左下角区域分为左右两部分：左侧为学习笔记面板，学生可在此记录学习心得；右侧为系统推荐的教学资源，内容包含教材页码与视频链接，点击即可跳转观看，观看结束后系统会弹出满意度评价问卷以收集反馈。右下角为留言板区域，学生可搜索并查看同班同学列表，选择特定对象或切换至全班通知模式后发送留言；消息气泡根据发送者角色以不同颜色区分，便于快速识别对话来源。

5.8.3 教师资源页仪表板

教师端采用上下分区布局，如图 5-5所示。上半部分的全班学习进度表格采用虚拟滚动技术，即使班级人数较多也能保持较流畅的交互，表格中每名学生的各课程完成状态以 $\sqrt{\times}$ 标记，并汇总整体完成率；教师可按学校、年级、班级维度进行筛选，快速定位需要关注的学生。下半部分采用左右分栏结构：左侧面板以百分比进度条展示班级典型错误分布，帮助教师把握整体错误特征；右侧为学习记录面板，教师可搜索学生列表、

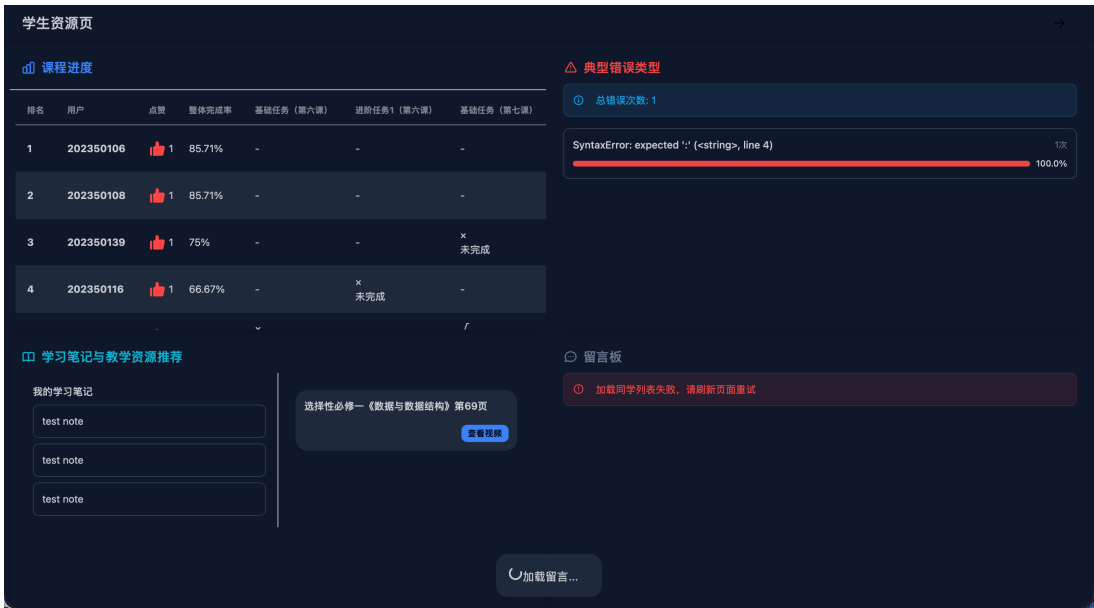


图 5-4 学生资源页仪表盘

查看历史消息记录，并以教师身份发送回复、留言或通知，未读消息数量会在学生列表中实时提示。

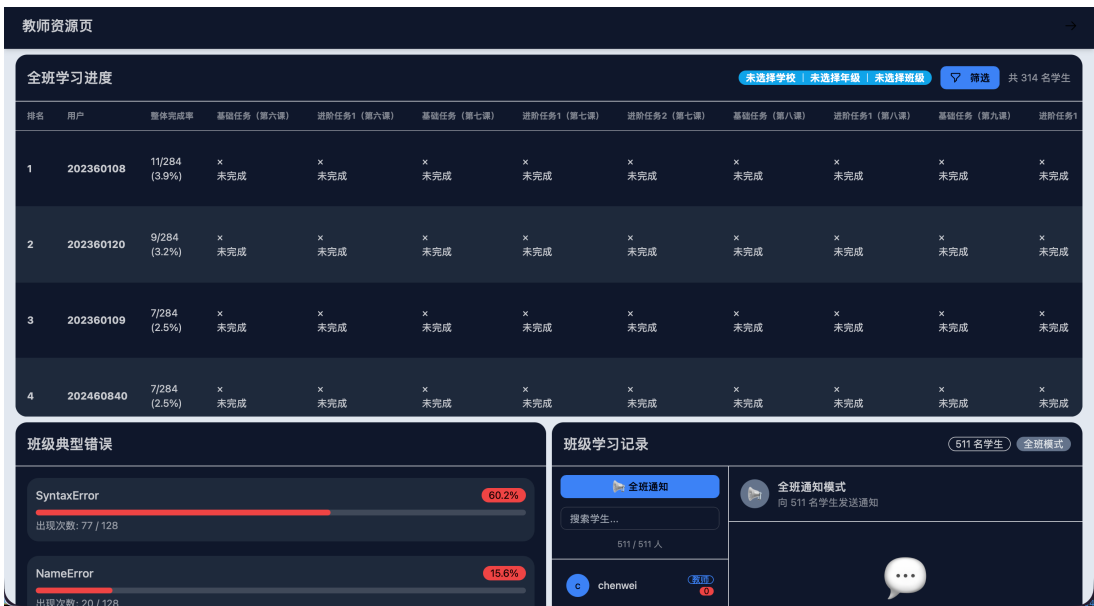


图 5-5 教师资源页仪表盘

5.8.4 教师资源管理页面

资源页中的标签页支持切换至”教学资源”板块，如图 5-6所示。该板块提供”所有资源”与”我的资源”两个视图：前者展示当前权限范围内可见的全部教学资源，后者仅展示教师本人创建的资源，便于教师快速定位自己维护的内容。每个资源卡片展示名

称、关联知识点、权限级别（所有人/本学校/仅班级）、创建者、观看次数与创建时间等元信息。

”添加资源”按钮用于创建新资源，填写名称、示范题目、资源代码、教材页码、视频链接与知识点后保存。对于已创建的资源，编辑与删除操作也可在此完成。资源权限级别由前端统一选择器控制，系统据此过滤不同权限下的资源可见范围，从而实现分层教学资源的精细化管理。



图 5-6 教师资源管理页面

6 系统测试与调试

6.1 测试环境

测试环境配置如表 6-1 所示。

表 6-1 测试环境配置

类别	配置
前端环境	Nuxt 3 + Node.js (pnpm)
后端环境	FastAPI + Python 3.14 + Uvicorn
数据库	SQL Server + SQLAlchemy ORM
缓存	Redis
模型接口	DashScope (通义千问) + OpenAI + 智谱 GLM
对象存储	阿里云 OSS
部署服务器	8 核 16G (生产环境)

测试环境配置以贴近真实部署为取向，让结论有切实参考。前端侧，ESLint 管代码规范，构建时开启 TypeScript 严格模式保类型安全与可维护性。后端侧，pytest 做回归验证，逐层核对接口逻辑、鉴权和业务调度。数据库与缓存着重读写验证，尤其在意高访问量下的稳定性。模型接口测试则把重心放在智能辅学流程本身上，在实际调用中检验响应的完整性和教学可用性。

本章测试并不只关注单个接口能否返回成功状态，而是围绕“登录认证—课程访问—代码提交—行为记录—教师反馈”这一学习回路展开。后端自动化测试主要使用 SQLite 与 FakeRedis 构造轻量测试环境，避免测试过程依赖外部 SQL Server 与 Redis 服务；前端验证则结合构建检查、页面截图和浏览器控制台日志，判断界面渲染、路由跳转与接口调用是否符合预期。对于大模型相关功能，考虑到远程模型接口存在网络波动和服务排队等因素，测试重点放在请求路径、提示词选择、输出格式约束和异常兜底，而不是简单以单次响应时长作为唯一依据。

6.2 功能测试

功能测试覆盖系统核心业务模块，测试结果如表 6-2 所示。

功能测试基于 pytest 框架编写，后端测试覆盖认证、权限控制、在线编程执行、消息互动与资源管理等多个模块。测试过程中借助 FastAPI 内置的 TestClient 发起接口调用，并在每次测试前完成登录态模拟，用以验证鉴权机制是否生效。

表 6-2 功能测试结果

测试编号	测试模块	测试内容	预期结果	实际结果
T1	登录认证	教师账号密码登录	登录成功并返回 JWT	通过
T2	登录认证	学生错误课程码登录	返回 403 禁止访问	通过
T3	登录认证	未登录访问受保护接口	返回 401 未授权	通过
T4	在线编程	提交正确代码 <code>print(1+1)</code>	返回输出 2	通过
T5	在线编程	提交危险代码 <code>import os</code>	返回 SecurityError	通过
T6	在线编程	提交无限循环代码	返回迭代次数超限错误	通过
T7	报错分析	提交含语法错误的代码	返回错误解释与定位	通过
T8	消息互动	教师发送消息给学生	学生可正常接收	通过
T9	消息互动	查询师生对话记录	返回完整对话历史	通过
T10	资源管理	更新资源访问权限级别	权限更新成功	通过
T11	资源管理	查询资源权限设置	返回当前权限值	通过
T12	课程管理	访问课程内容接口	返回课程结构数据	通过

功能测试围绕核心业务流程组织，避免停留在零散的界面点击检查。每项测试既验证单点功能可用性，也追溯前后流程的完整衔接。以在线编程模块为例：确认代码可执行只是入口条件，后续还需核对运行记录是否准确写入数据库、错误信息是否按约定格式返回，以及前端页面能否正确解析并展示这些结果。

从测试脚本对应关系看，认证类用例主要落在 `test_auth_and_permissions.py` 中，覆盖教师登录、学生课程码校验、未登录访问拦截和登录后身份检查；在线编程用例集中在 `test_platform_run.py`，分别验证正常输出、危险导入拦截和无限循环限制；消息与资源权限相关验证则由 `test_comments_and_resources.py` 承担。这样的拆分方式与系统模块边界基本一致，便于在某一类功能发生修改后执行定向回归测试。

危险代码执行测试体现了系统在多层安全防护上的设计。首先通过 AST 语法树分析，检查代码中是否存在危险导入（如 `os`、`sys`、`subprocess` 等模块）；其次在执行过程中使用 `sys.settrace` 监控迭代次数，防止无限循环占用服务资源。测试结果表明，这两种机制均能有效拦截危险代码。

智能辅学功能的测试重点在于输出结构的规范性，而非仅确认模型“返回了内容”。在教育场景中，模型作答是否切题、条理是否清晰，以及能否适配中小学生的认知水平，都是必须纳入考量的维度。测试覆盖自由问答、代码解释、报错解析、问题分解、抽象建模和算法设计引导六种场景，用于验证系统提示词约束与输出格式机制的有效性。

6.3 前端构建与代码质量检查

前端侧 pnpm 管包，ESLint 查规范，构建时走 TypeScript 严格模式保类型安全。

执行 `pnpm run build` 后，系统成功完成生产版本构建，输出文件总大小约 137 MB（压缩后约 31.8 MB）。构建产物包含服务器端渲染所需的 Nitro 服务器文件，以及各页面的按需加载分块。

代码质量检查发现部分组件存在未使用变量、事件未声明等问题，主要集中在 `Chat.vue`、`KnowledgePointSelector.vue` 等组件中。这些问题属于代码规范层面，尚未影响核心功能运行，可在后续版本更新中逐步处理。

页面级验证主要保存于 `output/playwright/` 目录，包括登录页、学生端主页和事件记录页等截图。截图材料用于证明关键页面能够完成基础渲染，控制台日志则用于定位接口联调问题。例如，当后端服务未启动或端口配置不一致时，日志中会出现 `ERR_CONNECTION_REFUSED`。这类记录不能直接说明前端页面实现错误，但提示端到端验证必须同时确认前端地址、后端端口与运行环境变量。基于此，本文将前端构建检查、截图复核和接口联通性检查分开记录，避免把环境问题误判为功能缺陷。

6.4 性能测试

性能测试覆盖接口响应时间、代码运行耗时、大模型调用时延以及多用户并发访问等维度。

6.4.1 后端服务性能配置

后端服务采用 Uvicorn 作为 ASGI 服务器，生产环境配置如下：

- 工作进程数：12 workers（基于 8 核 CPU 配置）
- 事件循环：uvloop（高性能事件循环实现）
- HTTP 解析：httptools（高性能 HTTP 解析器）
- 并发连接限制：1000
- 请求处理上限：每 worker 10000 请求后重启
- Keep-alive 超时：5 秒

这台 8 核 16G 的服务器上，该配置能把硬件资源吃透，并发场景里站得住。

6.4.2 接口响应性能

针对高频访问接口，系统提供 `backend/tools/perf_sample.py` 作为可复现实验脚本。该脚本在本地创建 SQLite 数据库并注入 FakeRedis，随后通过 `TestClient` 重复调用健康检查、登录态校验和在线代码运行接口，最终输出样本数、成功数、平均响应时间、P95 响应时间和最大响应时间等指标。与直接观察浏览器网络面板相比，这种方式更适合论文记录：既能固定测试数据和认证状态，又能减少外部数据库、缓存服务和网络传输对结果的干扰。

性能结果的解释需要区分三类耗时来源。纯后端逻辑耗时，如 `/health` 和 `/users/check`，主要反映路由处理、鉴权解析与依赖注入成本；业务执行耗时，如 `/platform/run`，除接口框架开销外，还包含代码安全检查、执行线程启动和输出捕获；外部服务耗时，如大模型调用、对象存储访问和云端数据库查询，更容易受网络状态影响。因此，本文在性能分析中重点使用平均值和 P95 指标观察稳定性，而不以单次最快或最慢结果作为整体性能结论。

6.4.3 代码执行性能

在线编程模块的性能表现直接影响学生学习体验。测试表明，对于典型的 Python 基础代码（如输入输出、条件判断、循环结构），执行耗时均在 100ms 以内。当代码进入较复杂计算时，用时自然拉长，但平台通过超时控制（默认 5 秒）与迭代次数限制（默认 50000 次）来兜住底线。

6.4.4 大模型调用性能

智能辅学功能依赖远程大模型接口，响应时延受网络状况和模型服务负载左右。实测中，DashScope 接口的响应大多落在 1 至 3 秒，复杂推理时会更久。为此系统走流式响应（Streaming Response），让输出逐字吐出来，而不是攒齐了再一股脑返回。

需要说明的是，大模型调用性能与常规后端接口不同。普通接口的主要瓶颈通常来自数据库访问和业务逻辑执行，而模型接口还受到提示词长度、输出 token 数、服务商排队和跨网络传输影响。因而在课堂使用场景下，系统更关注首字响应时间和输出连续性：只要学生能够及时看到模型开始生成内容，等待完整答案的主观延迟会明显降低。流式响应机制正是围绕这一体验目标设计的。

6.5 安全测试

安全性测试从认证安全、代码执行安全与数据访问安全三个维度展开。

6.5.1 认证安全测试

JWT 认证机制的测试覆盖以下场景：

- Token 有效性校验：过期或经篡改的 Token 均被拒绝访问
- 角色权限隔离：学生身份无法调用教师专用接口
- HttpOnly Cookie 防护：阻止客户端脚本读取 Token，降低 XSS 攻击面

6.5.2 代码执行安全测试

在线编程模块的安全测试验证了以下防护机制：

- 危险导入拦截：成功拦截 `os`、`sys`、`subprocess` 等模块的导入尝试
- 无限循环检测：通过迭代次数监控，防止死循环占用服务资源
- 执行超时控制：超过预设时间后强制终止代码执行
- 系统调用限制：禁止代码中调用系统级危险操作

6.5.3 数据访问安全测试

数据库访问安全通过 SQLAlchemy ORM 的参数化查询机制防范 SQL 注入攻击。测试重点在于确认用户输入不会被直接拼接进 SQL 字符串，而是经由 ORM 查询构造与参数绑定过程进入数据库操作，从而降低注入风险。

除上述三类安全测试外，系统还保留异常日志和错误返回结构，用于支持后续问题追踪。对于学生端代码执行失败、接口鉴权失败和前端请求失败等情况，测试不只检查

是否“报错”，还需要确认错误类型、状态码和提示信息是否便于前端识别。这个要求在教学平台中较为重要：若错误只表现为通用失败提示，学生和教师都难以判断问题来自代码、权限还是系统环境。

6.6 测试结果分析

测试结果表明，平台主要功能流程已可贯通运行，整体架构设计与模块划分具备基本可行性。

综合各类测试结果可以看出，系统验证更接近工程可用性测试，而非严格意义上的教学效果实验。换个角度看，本章证明的是平台能够支撑“登录、学习、提交、反馈、记录”这些关键动作连续发生；至于这些功能是否显著提升学生编程能力，还需要后续课堂实验和对照数据进一步论证。将这两类结论区分开，可以避免把系统可运行性直接等同于教学有效性。

6.6.1 功能完整性

核心功能流程——登录认证、课程访问、在线编程、报错解析、消息互动和资源管理——跑通了测试。前后端分离架构下，接口通信稳得住，数据格式对得齐，错误处理相对完整。认证鉴权那边，学生和教师的访问边界也划得清楚。

6.6.2 系统稳定性

表 6-2 中与代码执行安全相关的用例都跑通了，危险导入、无限循环、执行超时这几类异常能被基础防护机制兜住。Redis 缓存策略卸下了数据库的一部分访问压力，高频接口响应跟着提了一截。服务进程侧也配上了自动重启和并发上限，让系统不至于在意外中断后躺平。

6.6.3 教学适用性

智能辅学模块覆盖的六种功能场景，集成测试已经跑过了。系统能根据任务类型匹配合适的提示词模板，并把模型输出约束到中小学生能消化的范围里。消息互动模块替师生搭了一条正式沟通路径，学习行为记录模块则让教师能看清每个学生的学习状态，教学干预也有了数据依据。

6.6.4 待改进方向

现阶段测试集中在功能可用性与基础安全性层面，尚未建立大规模自动化测试覆盖与性能压力测试数据。后续可从以下方向继续完善：

- 引入端到端测试（E2E Testing），覆盖完整学生学习流程

- 接入自动化测试工具，建立持续集成流水线
- 实施多用户并发压力测试，评估系统在课堂环境中的实际承载能力
- 收集真实教学场景使用数据，对智能辅学效果开展量化评估

若推进到教学实验评估阶段，成熟量表与任务型指标可以搭着使。例如，用计算思维量表看学生在抽象、分解和算法表达上的变化^[1]，再拿学习行为日志对照不同提示策略下阶段的迁移特征^[18]。也有研究在自然科学课程里试过概念图嵌入式学习活动，结果提示结构化知识组织能撬动学习绩效，这个思路搬到编程课程的知识点可视化设计里同样值得一试^[15]。

眼下还没有成体系的课堂实验数据和大规模压测数据，所以本章分析集中在系统实现与功能可用性上，不往上推导教学效果。后面如果真进了课堂，把任务完成率、纠错效率、学习兴趣和自我效能感量表一铺开，对平台教学支持的评估才算有说服力^[5]。

7 结论

本文面向中小學生编程学习场景，设计了一套基于大模型的智能编程学习平台并完成了系统实现。架构上走前后端分离方案，课程管理、任务学习、在线编程、消息互动与资源管理是底座模块，智能辅学与行为记录功能接在它们上面。智能辅学模块内部，本文紧扣教学目标，对场景划分、提示约束和输出组织做了适配，让各项功能在学习流程中自然衔接在一起。

与把大模型简单塞进聊天框的做法不同，本文的关注点落在教学场景适配、系统工程约束和教师可控性上。对中小學生的编程辅学而言，重点不是模型本身有多强，而是它的输出能不能被学生理解、被教师追踪、被系统干预。正是从这一点出发，本文构建的是教师牵线、学生动手、大模型辅助的三方协作模式，而不是把教师晾在一边的自动化方案。

从已完成的工作来看，当前能确认的结论集中在系统设计合理、核心功能可运行这两点。至于平台对学生编程能力、学习兴趣、自我效能感和长期行为的实际带动效果，还得靠后续课堂实验、对照测试和过程数据来检验。延伸方向包括课程知识库的检索增强、学习者建模、提示策略评估、教师干预策略设计和教学实验验证等。

参考文献

- [1] TSAI M J, LIANG J C, HSU C Y. The computational thinking scale for computer literacy education[J]. Journal of Educational Computing Research, 2021, 59(4): 579-602.
- [2] WANG S, XU T, LI H, et al. Large language models for education: a survey and outlook [EB/OL]. 2024. DOI: 10.48550/arXiv.2403.18105.
- [3] RAIHAN N, SIDDIQ M L, SANTOS J C S, et al. Large language models in computer science education: a systematic literature review[C/OL]//Proceedings of the 56th ACM Technical Symposium on Computer Science Education V.1. ACM, 2025: 938-944. DOI: 10.1145/3641554.3701863.
- [4] YIM I H Y, SU J. Artificial intelligence (AI) learning tools in K-12 education: a scoping review[J/OL]. Journal of Computers in Education, 2024. DOI: 10.1007/s40692-023-00304-9.
- [5] TANG B, LIANG J, HU W, et al. Enhancing programming performance, learning interest, and self-efficacy: the role of large language models in middle school education[J/OL]. Systems, 2025, 13(7): 555. DOI: 10.3390/systems13070555.
- [6] PITTS G, HRIDI A P, LEKSHMI-NARAYANAN A B. A survey of LLM-based applications in programming education: balancing automation and human oversight[EB/OL]. 2025. DOI: 10.48550/arXiv.2510.03719.
- [7] CHEN L, XIAO S, CHEN Y, et al. ChatScratch: an AI-augmented system toward autonomous visual programming learning for children aged 6-12[C/OL]//Proceedings of the CHI Conference on Human Factors in Computing Systems. ACM, 2024: 1-19. DOI: 10.1145/3613904.3642229.
- [8] MA Q, SHEN H, KOEDINGER K R, et al. How to teach programming in the AI era? using LLMs as a teachable agent for debugging[C/OL]//Artificial Intelligence in Education. Springer, 2024: 265-279. DOI: 10.1007/978-3-031-64302-6_19.
- [9] GONZALEZ-MALDONADO D, LIU J, FRANKLIN D. Evaluating GPT for use in K-12 block based CS instruction using a transpiler and prompt engineering[C/OL]//Proceedings of the 56th ACM Technical Symposium on Computer Science Education V.1. ACM, 2025: 388-394. DOI: 10.1145/3641554.3701910.

- [10] PIRZADO F A, AHMED A, MENDOZA-URDIALES R A, et al. Navigating the pitfalls: analyzing the behavior of LLMs as a coding assistant for computer science students—A systematic review of the literature[J/OL]. IEEE Access, 2024, 12: 112605-112625. DOI: 10.1109/ACCESS.2024.3443621.
- [11] HUMBLE N. Risk management strategy for generative AI in computing education: how to handle the strengths, weaknesses, opportunities, and threats?[J/OL]. International Journal of Educational Technology in Higher Education, 2024, 21(1). DOI: 10.1186/s41239-024-00494-x.
- [12] TLILI A, SHEHATA B, ADARKWAH M A, et al. What if the devil is my guardian angel: ChatGPT as a case study of using chatbots in education[J/OL]. Smart Learning Environments, 2023, 10(1). DOI: 10.1186/s40561-023-00237-x.
- [13] YAN Y M, CHEN C Q, HU Y B, et al. LLM-based collaborative programming: impact on students' computational thinking and self-efficacy[J/OL]. Humanities and Social Sciences Communications, 2025, 12(1): 149. DOI: 10.1057/s41599-025-04471-1.
- [14] CHEN S J, SHAN X, LIU Z M, et al. Employing large language models to enhance K-12 students' programming debugging skills, computational thinking, and self-efficacy[J/OL]. Educational Technology & Society, 2025, 28(2): 259-278. DOI: 10.30191/ETS.202504_28(2).TP01.
- [15] HWANG G J, YANG L H, WANG S Y. A concept map-embedded educational computer game for improving students' learning performance in natural science courses[J]. Computers & Education, 2013, 69: 121-130.
- [16] 姚佳佳, 李艳, 潘金晶, 等. 同伴对话反馈对大学生在线深度学习的影响研究[J]. 华东师范大学学报（教育科学版）, 2022, 40(3): 112-126.
- [17] LI M, KAMARAJ A V, LEE J D. Modeling trust dimensions and dynamics in human-agent conversation: a trajectory epistemic network analysis approach[J/OL]. International Journal of Human-Computer Interaction, 2023. DOI: 10.1080/10447318.2023.2201555.
- [18] SUN D, ZHU C, XU F, et al. Transitioning from introductory to professional programming in secondary education: comparing learners' computational thinking skills, behaviors, and attitudes[J/OL]. Journal of Educational Computing Research, 2023. DOI: 10.1177/07356331231204653.
- [19] JEON J B, LEE S. Large language models in education: a focus on the complementary relationship between human teachers and ChatGPT[J/OL]. Education and Information Technologies, 2023, 28(12): 15873-15892. DOI: 10.1007/s10639-023-11834-1.

- [20] TOURETZKY D, GARDNER-MCCUNE C, MARTIN F, et al. Envisioning AI for K-12: what should every child know about AI?[C/OL]//Proceedings of the AAAI Conference on Artificial Intelligence: Vol. 33. 2019: 9795-9799. DOI: 10.1609/aaai.v33i01.33019795.
- [21] 教育部基础教育教学指导委员会. 中小学生生成式人工智能使用指南（2025 年版）[Z]. 2025.
- [22] 中国自动化学会, 中国人工智能学会, 中国科学院大学人工智能学院, 等. 关于举办全国中小学人工智能探究性学习训练营（2025 北京）的通知[Z]. 2025.
- [23] 北京师范大学智能技术与教育应用教育部工程研究中心, 北京市数字教育中心. 人工智能赋能基础教育应用蓝皮书（2025 年）[R]. 发布单位, 2025.
- [24] BEALE R. The revolution has arrived: what the current state of large language models in education implies for the future[EB/OL]. 2025. DOI: 10.48550/arXiv.2507.02180.

致谢

我要向我的导师匡振中教授致以最诚挚的感谢，感谢他在本论文完成过程中给予的持续指导、富有洞察力的建议和坚定不移的支持。他严谨的治学态度和慷慨的鼓励激励着我不断探索项目的边界，能在他的指导下完成研究是我莫大的荣幸。

这个毕业设计——一个以大型语言模型为动力的在线编程教育平台——如果没有在整个系统开发过程中协助我的工程师的实践指导，是不可能实现的。我真诚地感谢动手实践的技术咨询、耐心的调试环节，以及真实世界的工程视角，它们极大地塑造了最终的实施。

我也向帮助我进行线下工作的老师和同学表示由衷的感谢。他们在用户访谈、评估实验和实地测试中的慷慨帮助提供了必要的反馈，直接提高了平台的可用性和有效性。

最后，我要感谢我的家人无条件的关爱和不断的鼓励。他们对我的信任一直是我最大的力量源泉。

附录

附录 A 项目技术栈与系统结构

本课题实现了一个基于 Nuxt 3 + FastAPI 的编程教育平台，覆盖课程管理、学生/教师角色区分、聊天交互与事件追踪。技术栈选型上：前端走 Nuxt 3（Vue 3）、Pinia、TypeScript、Tailwind CSS + DaisyUI，配上 Axios、ECharts、CodeMirror 和 Tiptap 搞定交互展示；后端由 FastAPI、Uvicorn、SQLAlchemy、pymssql、Redis、JWT 和 passlib[bcrypt] 撑起接口与数据层；部署运维这边，Docker、PM2、ESLint、Pytest 外加性能监控模块一起扛交付和验证，整套方案结构还算清爽。

从目录组织看，frontend/ 主要承载页面、组件、状态管理和静态资源，backend/ 集中放置 API 路由、数据库模型、公共模块与工具模块；输出目录 output/playwright/ 用于保存测试截图和日志。这种组织方式便于后续功能实现、联调验证和实验材料整理，也能在需要时快速定位模块。

结合近两年的综述与实证研究可以看出，LLM 在编程教育中的价值通常体现为“过程性支持”而非“替代性教学”：教育场景中的综述普遍将其定位为反馈增强、学习支架与课程辅助工具，强调与教师决策协同使用^[2-3,6]；但针对编程学习助手的整体性证据也反复提示，模型输出可能出现不稳定或不完全可靠的情况，因此课程设计需要同步配置验证、追问与人工复核机制^[10,19]。在 K-12 语境下，相关范围综述同样指出，工具选择应与学习者年龄、任务粒度和课堂组织方式相匹配，避免“技术先行、目标滞后”的实施路径^[4]。上述结论与本项目采用的“前后端分层 + 测试留痕 + 过程可追溯”结构基本一致，可为后续课堂化部署提供方法学依据。

附录 B 后端接口与数据库模型摘要

后端路由按业务场景分组，整体上可以分为两类：

- 公开接口：/health、/users、/classes
- 认证平台接口：/platform/contents、/platform/courses、/platform/records、/platform/tasks、/platform/knowledges、/platform/recommendations、/platform/suggestions、/platform/resources、/platform/comments、/platform/likes

数据库模型包括 Users、Tasks、Contents、Courses、Knowledges、Recommendations、Records、Suggestions、Classes、Resources、Comments、

`CommentReadStatus` 和 `Likes`。这些表结构分别对应用户、课程、学习行为记录、反馈与互动等核心业务，为平台保持数据一致性提供了基础。

接口与数据层设计这块，近期研究对“可解释流程”和“教师在环”的呼声趋同：编程教育里，LLM 更适合干提示、纠错建议和阶段反馈，评分裁决、学情诊断、教学意图解释得交回教师或课程规则约束^[3,8,19]。K-12 的实验和课堂报告也印证了，提示工程、任务拆解和中间产物检查能拉高交互有效性，但需要配套行为记录与结果审计来压误导风险^[5,9]。因此，附录里的 `Records`、`Suggestions`、`Comments` 几个模型不只是跑业务，更是在为“过程留痕 + 教学回溯”铺数据底座，跟风险治理里可追踪、可追责的原则一拍即合^[10-12]。

附录 C 性能采样脚本

为便于论文实验数据收集，后端提供了接口性能采样脚本 `backend/tools/perf_sample.py`。该脚本在本地以 SQLite 和 FakeRedis 复现实验环境，对多个接口进行重复采样，并将结果输出为 JSON，便于直接整理为论文表格。

脚本采样的接口包括：`GET /health`、`GET /users/check`、`POST /platform/run (print)`。输出指标包括样本次数、成功次数、平均响应时间、P95 响应时间和最大响应时间。

相关文献也支持将性能指标与学习支持质量放在一起考察。LLM 介入编程教学后，响应时延与交互稳定性会直接影响学生持续提问和迭代调试的意愿，低抖动、可复现实验流程是必要前提^[6,8]。此外，系统综述普遍建议将模型正确性、反馈可操作性与失败案例占比纳入过程评估，而不只看单次成功率^[3,10]。学校场景研究也显示，经任务约束与提示优化后，模型在特定编程活动中的辅助效果更稳定，但结果可靠性的核验步骤不能省^[5,9]。基于这一结论，本课题在性能采样之外保留 JSON 明细，便于后续按实验批次回溯接口耗时、反馈质量和学习行为之间的关联。

其核心流程可概括为：

- (1) 初始化测试环境变量，并准备 SQLite 数据库和 FakeRedis；
- (2) 载入最小测试数据，同时覆盖 FastAPI 的数据库依赖；
- (3) 通过 `TestClient` 调用接口，统计响应耗时；
- (4) 将结果写入 JSON 文件，供论文实验部分引用。

附录 D 测试截图与日志摘要

项目在 `output/playwright/` 下保存了自动化测试的截图与日志，可作为系统验证材料，具体包括：

- ui_login_local.png 和 ui_login_cloud.png，对应登录页面截图；
- ui_stu.png，对应学生端主页截图；
- ui_event_records.png，对应事件记录页面截图；
- console-*.log，记录了 Playwright 控制台日志以及前后端联调时的连接与抓取信息。



图 7-1 本地环境登录页面测试截图（重采样）

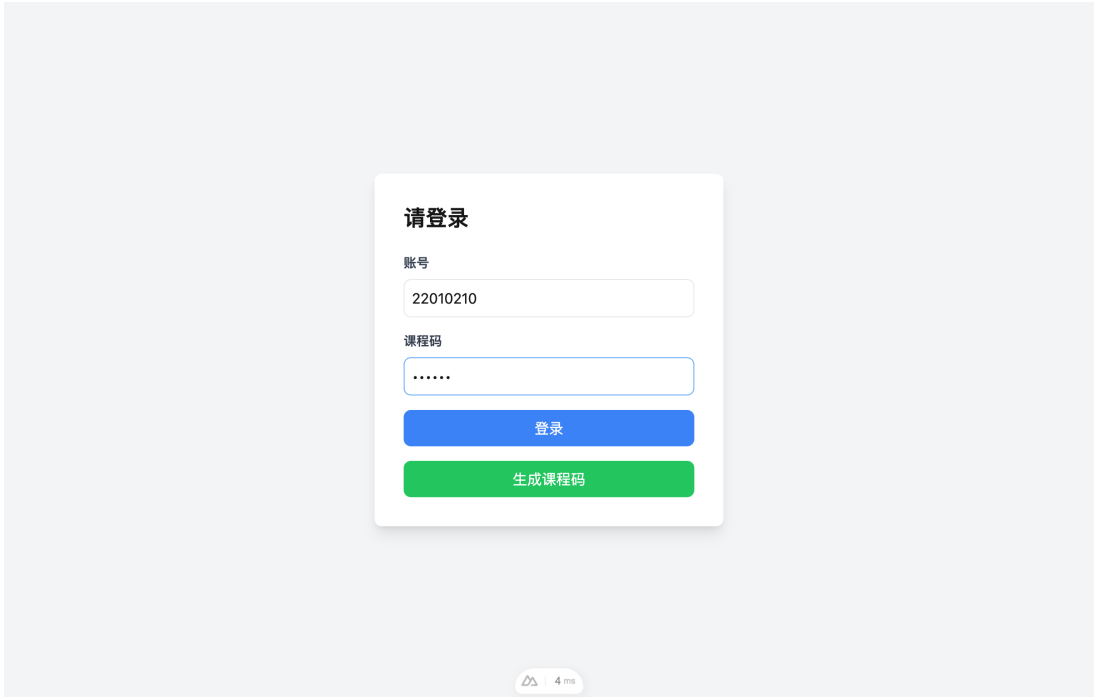


图 7-2 登录表单填写状态截图（重采样）



图 7-3 学生端主页测试截图（重采样）



图 7-4 事件记录页面测试截图（重采样）

请登录

账号

课程码

登录

生成课程码

图 7-5 云端环境登录页面测试截图

从以上材料可以看出，系统界面、认证流程、事件追踪页面和自动化测试流程均已完成，验证流程已基本贯通。

对照教育领域关于生成式 AI 的已有研究，测试截图与日志除用作可用性证明外，还应承担教学风险提示功能。文献普遍认可其在学习支持和资源获取上的效率优势，但也持续报告了幻觉、过度依赖、学术诚信和隐私边界等问题，需要通过日志抽样、异常

对话复核和规则提示加以约束^[11-12,19]。对于 K-12 与初学者场景，范围综述与系统综述均建议在平台端保留可审计证据，并将人机协同规范显式纳入课程说明与评测标准^[2-4]。因此，`output/playwright/` 下的截图和控制台记录既是工程验收材料，也可作为论文中技术验证与教学治理的协同证据。